

Звіт

Лабораторна робота №3 (a,b)

“Паралельне мультипоточне програмування”

Виконала студентка групи ІПС-22

Чергінець Юлія Дмитрівна

1. Мета роботи

Опанувати принципи розробки багатопотокових застосунків, засвоїти методи розділення обчислювальних ресурсів та синхронізації. Практично реалізувати імітаційне моделювання складної системи з обробкою великих обсягів даних, застосовуючи об'єктно-орієнтований підхід, покриття юніт-тестами та графічний інтерфейс для взаємодії.

2. Опис розробленої системи Для виконання обох частин лабораторної роботи (3a та 3b) було розроблено програмний модуль TradeSim Backtest Engine. Це система алгоритмічного бектестингу, яка імітує поведінку торгового бота на історичних ринкових даних.

Система генерує великі масиви даних (котирування) і виконує ресурсомісткі математичні обчислення для розрахунку торгових індикаторів (аналіз попередніх 100 свічок із використанням тригонометричних та логарифмічних функцій). На основі цих індикаторів система імітує відкриття та закриття торгових позицій, підраховуючи фінальний прибуток (PnL) та кількість угод.

3. Об'єктно-орієнтований дизайн та архітектура Архітектура побудована мовою TypeScript у середовищі Node.js. Основна логіка інкапсульована у класі BacktestService. Методи класу чітко розділяють відповідальність:

- generateMockData — генерація вхідних даних (використовує оптимізовану структуру Float64Array).
- runSequential — класичний однопотоковий алгоритм.
- runParallel — багатопотоковий алгоритм, який відповідає за оркестрацію робочих потоків та агрегацію результатів.

Система інтегрована у загальний проєкт TradeSim, де вже реалізовано 12 патернів проєктування (згідно з Лабораторною №2), що забезпечує масштабованість та відповідність принципам SOLID.

4. Реалізація мультипоточності Для розпаралелювання обчислень було використано модуль worker_threads.

- **Послідовна версія:** Алгоритм проходить циклом по масиву в головному потоці (Main Thread), що блокує виконання інших операцій до завершення обчислень.

- **Паралельна версія:** Головний потік нарізає вхідний масив на рівні фрагменти (chunks) та асинхронно розподіляє їх між заданою кількістю робочих потоків (від 2 до 8). Для синхронізації та очікування завершення роботи всіх потоків використовується Promise.all.

Під час розробки було вирішено архітектурну проблему межових станів (Data Overlap). Оскільки індикатору для розрахунку потрібна історія за 100 попередніх періодів, при простому розрізанні масиву виникала розбіжність у розрахунках. Проблема вирішено шляхом зміщення індексу початку фрагмента (startIdx -= 100) для кожного потоку, крім першого. Це забезпечило потокам необхідний історичний контекст без дублювання виконання угод.

5. Покриття юніт-тестами Функціональність покрита автоматизованими тестами за допомогою фреймворку Vitest. Написані тести перевіряють:

- Правильність генерації структур даних заданого розміру.
- Стійкість алгоритму до некоректних даних (обробка порожнього масиву паралельними потоками).
- Ідентичність математичних результатів. Тест підтверджує, що фінансовий результат (profit) та кількість угод (trades), обчислені паралельною мультипоточною версією, збігаються з результатами послідовної версії в межах допустимої статистичної похибки (викликані розривом активних позицій на межах потоків).

6. Графічний інтерфейс користувача Для візуалізації та керування системою розроблено клієнтську частину на базі React (компонент BacktestPanel). Інтерфейс дозволяє користувачу:

- Динамічно обирати обсяг вхідних даних (500 тис., 1 млн або 5 млн записів).
- Вказувати кількість робочих потоків (2, 4 або 8).
- Запускати тестування та отримувати результати у вигляді зручних метрик із розрахунком коефіцієнта прискорення. Взаємодія між клієнтом та сервером реалізована через REST API.

7. Вимірювання часу виконання та аналіз результатів Час виконання вимірювався за допомогою performance.now() у мілісекундах. Було

проведено серію тестів на різних обсягах даних з використанням 4 робочих потоків.

Розмір масиву (записів)	Час (Послідовно)	Час (4 Потоки)	Прискоренн я
1 000 000	1927 мс	1024 мс	у 1.88х разів
5 000 000	9496 мс	3944 мс	у 2.41х разів

Аналіз: результати демонструють, що на відносно невеликих масивах даних (1 млн записів) прискорення не є лінійним через накладні витрати ресурсів на створення потоків (Thread Overhead) та передачу повідомлень між ними. Проте, при збільшенні навантаження до 5 млн записів ефективність мультипоточності суттєво зростає (прискорення сягає 2.41х), оскільки частка корисних обчислень значно перевищує витрати на синхронізацію.

8. Документація коду Увесь програмний код модулів бектестингу задокументовано згідно зі специфікацією JSDoc. За допомогою інструмента TypeDoc (аналог Doxygen для екосистеми TypeScript) було автоматично згенеровано статичний веб-сайт із документацією. Описано призначення класів, параметри методів, типи повернених значень та архітектурні рішення, пов'язані з потоками.

9. Висновки Під час виконання лабораторної роботи було успішно спроектовано та реалізовано систему алгоритмічного тестування, яка обробляє великі масиви даних. Практично доведено ефективність застосування мультипоточного підходу для обчислювально інтенсивних задач (CPU-bound) — пришвидшення роботи склало понад 2 рази. Забезпечено правильний об'єктно-орієнтований дизайн, вичерпне тестування, інтерактивний користувацький інтерфейс та технічну документацію.